

Unit 4: Software Testing & Maintenance

Software Testing Fundamentals

- **Definition:** Software testing is the process of exercising or evaluating a system to verify that it satisfies specified requirements or to identify differences between expected and actual results ¹. It is a critical quality-assurance phase that ensures a program meets user needs. In practice, testing can be manual or automated, and it encompasses *verification* (checking the product is built correctly) and *validation* (checking the right product is built) ².
- **Core Principles:** Testing aims to uncover bugs, faults, or defects (logic mistakes by developers) and to ensure software correctness, completeness, and quality ³. It is planned to cover all phases of development: e.g., during coding one creates unit tests, during integration one tests interfaces, and during maintenance one re-tests after fixes ⁴ ⁵. Effective testing lowers maintenance cost and increases user satisfaction by identifying errors (e.g. handling divide-by-zero in a calculator) early on.
- **Example:** An e-commerce checkout module might undergo extensive testing: unit tests for each function (e.g. calculate totals), integration tests with the payment gateway (to ensure correct data flow), and system tests with live databases. For instance, testers would validate that entering an invalid coupon code yields an error rather than a discount.
- **Takeaways:** Thorough testing is essential for software reliability. It should be systematic and planned, with clear objectives. Proper testing reveals errors **before** deployment, saving cost and effort. Remember that *verification* asks "Are we building the product right?" and *validation* asks "Are we building the right product?" ².

Test Planning

- **Definition:** A *test plan* is a formal document that specifies the scope, approach, resources, and schedule of testing activities. It defines *what* will be tested, *how*, *who* will do it, and *when*. In IEEE terms, it details the purpose, scope, and method of testing, the test items, features to be tested, the environment, and measurement techniques ⁶. Essentially, the test plan is an agreement between testers and stakeholders on the role of testing in the project.
- **Core Components:** A complete test plan typically includes:
- **Overview:** Objectives and functions of the software, and goals of testing (defining responsibilities, test environment, and information sources) ⁷.
- **Test Scope:** Features and feature combinations to be tested (and not tested), including interfaces and documentation ⁸.
- **Test Methodology:** Types of tests to be used (e.g. regression, stress) and selection of test data (boundary/null values), plus how results will be recorded ⁸.
- **Test Phases:** Description of unit, integration, system, and acceptance tests; who performs them; and data sources ⁹ ¹⁰.
- **Test Environment:** Hardware, software, tools, OS, and staffing required ¹⁰.
- **Schedule and Resources:** Detailed timeline of testing activities, dependencies, and assignments ¹¹.
- **Approvals and Distribution:** Signatories approving the plan and stakeholders receiving it ¹².

- **Planning Process:** Key steps are (1) set test-plan objectives aligned with software goals; (2) develop a *test matrix* mapping components to tests (ensuring every part has at least one test) ¹³; (3) define administrative details (scheduling, resources, and contingency for scope changes) ¹⁴; (4) compile the plan document in either informal (all details) or formal (summarized key points) style ¹⁵. For example, a banking app's test plan might list tests for login, transaction, and error-handling components, with a schedule allocating two weeks for unit tests and one week for system tests.
- **Example:** For an online ticketing system, a test plan would outline tests for seat selection, payment processing, and notification delivery. It would specify test cases (valid card, expired card, no seats left), the test environment (replica of production servers), and the schedule (unit tests in sprint 3, integration tests in sprint 4). The plan assigns roles (developers vs. QA team) and includes contingency for new feature requests.
- **Deep Takeaways:** A well-crafted test plan ensures systematic, comprehensive testing and clear accountability. It prevents gaps (no forgotten feature), overlaps (duplicate effort), and schedule delays. In exams, emphasize that *test planning is proactive*: it defines *how* defects will be found and fixed. A strong test plan acts as a blueprint for quality assurance and should be adaptable if project scope changes. Ultimately, it aligns the entire team on testing expectations and resource needs, reducing later confusion ⁶ ⁷.

Testing Strategies

- **Definition:** A testing strategy is a high-level approach describing which levels of testing to apply, in what order, and using which techniques. It is tailored to the project's nature (e.g. data-intensive vs. transaction-oriented) and provides a template for systematic testing ¹⁶.
- **Common Approaches:** Strategies often proceed "outward" from unit to integration to system testing ¹⁶. Testing is performed by developers and independent testers (ITG). The strategy must also accommodate debugging (since testing reveals bugs) ¹⁷. Examples include:
 - **Analytic Strategy:** Risk-based, assesses and prioritizes likely failure modes; gathers requirements/design information to focus tests ¹⁸.
 - **Model-based Strategy:** Uses real-world scenarios (e.g. workflow models) to drive functional tests, focusing on high-error domains ¹⁹.
 - **Methodical Strategy:** Employs checklists derived from requirements or design, covering functionality, reliability, usability, performance ²⁰.
- **Example:** In a medical records system, an analytic strategy might rank modules by patient-safety risk and prioritize tests accordingly.
- **Takeaways:** The right strategy depends on software type (e.g. web app vs. embedded system) and project constraints. Effective strategies ensure all aspects of the product (functional and non-functional) are tested. Remember that no single strategy fits all; often a combination (e.g. risk-based plus regression testing) is used.

Levels of Testing

- **Unit Testing:** Verifies individual modules or functions in isolation. Each "unit" (e.g. a single function or class) is tested with white-box techniques by the developer ²¹. This catches coding errors early. For instance, a function computing interest rates is fed various inputs (zero, negative, large) to ensure correct output. Unit tests are **repeated after every change** ²².
- **Integration Testing:** After unit tests pass, integration testing examines combined modules as a subsystem. The goal is to uncover interface and interaction errors. For example, after verifying

separate payment and order modules, integration tests ensure order details correctly flow into payment processing ²³ . Incremental integration (top-down or bottom-up) is often used to isolate faults: one module at a time is added and tested with stubs or drivers ²⁴ ²⁵ .

- **System Testing:** Involves testing the complete, integrated system as a whole. Here the software is combined with other elements (hardware, databases, networks, people) to form a computer-based system. IEEE defines system testing as testing a fully integrated system for compliance with requirements ²⁶ . It covers functional correctness and non-functional attributes (usability, performance, security). For instance, a web application is tested in the production-like environment with real user scenarios. System tests include recovery (forcing failures to test backups) and security tests (simulating attacks) ²⁷ ²⁸ .
- **Acceptance Testing:** The final level, often done by end-users or stakeholders to validate that requirements are met. It may involve user-acceptance tests, beta testing, or certification tests. (Note: the provided material briefly mentions acceptance but focuses more on earlier levels.)
- **Example:** In testing a smartphone app, a unit test would verify a single screen's logic; integration tests would check API calls and data flow between screens; system testing would validate the app on various devices, networks, and with real user accounts.
- **Takeaways:** Each level has a distinct purpose: unit testing finds coding bugs, integration testing catches interface issues, and system testing validates the entire application in context. In exams, be ready to contrast levels (e.g. "Unit tests ensure each component works; integration tests ensure components work together ²³ ; system tests ensure the complete product works in its environment ²⁶ "). Proper testing proceeds from small scale to full-system checks.

Testing Techniques

- **Black-Box Testing (Functional):** The tester checks functionality without any knowledge of internal code. Tests are based on specifications: given inputs, observe outputs. It is used to find interface and functional errors. For example, in black-box testing of a login form, one supplies various username/password pairs and verifies correct login or error message ²⁹ . Black-box tests catch errors like missing functions or incorrect output, and do not require coding skills ²⁹ . Techniques include equivalence partitioning and boundary-value analysis for input selection.
- **White-Box Testing (Structural):** The tester has full knowledge of the internal code and design. The goal is to exercise all code paths and logic branches. For example, with white-box testing of a sorting routine, the tester creates tests to cover every loop path and conditional. It requires understanding the program's structure, ensures all internal paths are executed, and is measured by code coverage ³⁰ ³¹ . White-box tests detect flaws in control flow, data structures, and logic.
- **Gray-Box Testing:** (Briefly) A mix of both; tester knows some internal workings (e.g. system architecture) but tests from a functional perspective. Often used in web apps.
- **Example:** Testing a calculator app: black-box tests verify correct results for inputs, while white-box tests ensure the code handles every operation branch (addition, subtraction, division by zero, etc.).
- **Takeaways:** Effective testing uses both techniques: black-box for validating requirements, and white-box for ensuring implementation quality. Many errors of omission (missing code) are found with white-box tests, while errors of commission (wrong output) are found with black-box tests.

Debugging

- **Definition:** Debugging is the process of locating, analyzing, and removing the causes of defects in software after they have been detected by testing ³² . It is performed once testing identifies a

failure. Debugging is not the same as testing: it focuses on *fixing* errors rather than *finding* them. Debuggers (tools) reproduce failure conditions, inspect program state, and let developers step through code to pinpoint the root cause ³³ .

- **Guidelines:** Effective debugging follows disciplined steps:
- **Understand the problem fully:** Before changing code, reproduce the defect and understand expected vs. actual behavior ³⁴ . Avoid “experimenting” with fixes that can introduce new bugs ³⁵ .
- **Check related code:** If an error appears in one module, examine other modules it interacts with (errors often cluster) ³⁶ .
- **Verify fixes:** After correcting a defect, ensure the new code itself has no errors by re-running regression tests ³⁷ .
- **Debugging Process:** A typical process is:
- **Defect Confirmation:** Verify the reported bug is genuine (can it be reproduced, and what was expected vs. actual?) ³⁸ .
- **Defect Analysis:** Use a debugger or logs to trace execution and find the root cause (step through code paths) ³⁹ .
- **Defect Resolution:** Make code changes to fix the problem once identified ⁴⁰ .
- **Retesting:** After fixing, rerun relevant tests to ensure the bug is gone and no new bugs were introduced ⁴¹ .
- **Debugging Strategies:** Because it is often time-consuming, developers use strategies to guide the process:
- *Brute Force:* Add extensive logging or use memory dumps to find the fault (used if nothing else works) ⁴² .
- *Induction:* Observe symptoms and generalize to locate error(s) (gather data, hypothesize causes, test hypotheses) ⁴³ ⁴⁴ .
- *Deduction:* List possible causes and eliminate them one by one until the actual cause remains (top-down reasoning).
- *Backtracking:* Start from the failure and trace backwards to find where state diverged.
- *Debugging by Testing:* Create additional targeted test cases to illuminate the error.
- **Example:** Consider a web app crashing on checkout. A tester provides details: “Error occurs when quantity = 0”. The developer reproduces the issue, sets a breakpoint in the checkout code, and steps through. Finding that the code does not handle zero quantity, they add a condition to display an error. They then rerun all checkout tests to confirm no new issues.
- **Deep Takeaways:** Debugging is systematic problem solving. It demands patience and methodical thinking. On exams, emphasize that debugging happens *after* testing and requires understanding the system’s internals. A key point is that you should *never rush* into code changes without analysis—trial fixes can make things worse ³⁴ . Good debugging finds the *root cause* (not just symptoms) so the defect is truly removed. Ultimately, debugging closes the testing loop: once bugs are resolved and re-tested, the software becomes more reliable.

Other Testing Concepts

- **Test Reports:** After test execution, a *test report* documents the outcomes: which tests passed/failed, actual versus expected results, and any defects found ⁴⁵ . It often includes statistics and analysis to inform stakeholders and guide maintenance. For example, recording failed cases helps reproduce and fix bugs later. A clear test report accelerates maintenance by showing exactly what went wrong ⁴⁵ .

- **V-Model:** A common testing model is the V-model, which pairs development phases with corresponding test levels (e.g. requirements ↔ acceptance tests, design ↔ system tests, coding ↔ unit tests). The V-model emphasizes that *testing activities should begin in early phases* and continue throughout development ⁴⁶. Its advantage is ensuring verification/validation occur continuously.
- **Object-Oriented Testing:** Testing O-O systems adds concerns like class-level testing, inheritance and polymorphism. Tests must cover class interfaces, interactions, and use-case scenarios specific to objects. For example, testing a subclass may require both unit tests for its own methods and integration tests to ensure it properly extends its parent class. (The provided material briefly notes O-O testing without deep detail.)
- **System/Acceptance Testing:** System testing has already been touched on; acceptance testing (often by the customer) ensures the software fulfills the agreed requirements. In practice, alpha/beta tests or user-acceptance sessions finalize this stage.

Software Maintenance

- **Definition:** Software maintenance is the process of modifying a software system **after delivery** to correct faults, improve performance or other attributes, or adapt it to a changed environment ⁴⁷. Unlike hardware, software doesn't physically wear out, but it must be "serviced" (updated and enhanced) to remain useful. IEEE's definition highlights correcting faults, enhancing features, and adaptation.
- **Purpose & Examples:** Maintenance ensures the software continues to meet user needs post-deployment. Key maintenance goals include:
- **Continuity of Service:** Fixing bugs and failures (e.g. a patch for a security hole), and handling changes in hardware/OS (e.g. updating an app for a new Android version) ⁴⁸. This is analogous to repairing a car when parts fail.
- **Mandatory Upgrades:** Implementing changes due to external factors like new regulations or standards. For instance, modifying a video-streaming app to block content in regions with censorship, or adding tax rule changes in an accounting system ⁴⁹.
- **Improvements (Perfective):** Enhancing functionality or performance at user request. For example, adding a new feature to customize reports in a payroll system or optimizing a slow algorithm ⁵⁰.
- **Future Maintenance Facilitation (Preventive):** Refactoring or reorganizing code and databases to make future changes easier ⁵¹. For example, restructuring a monolithic system into modular components so that new features can be added with less effort.
- **Analogy & Takeaway:** A helpful analogy is car maintenance: just as oil changes, part replacements, and upgrades (e.g. adding GPS) keep a car useful, software requires patches, updates, and refactoring to stay current ⁵². The key takeaway is maintenance is ongoing and often far more costly than initial development – emphasizing the need for maintainable design from the start.
- **Lehman's Laws:** Historical studies by Lehman observed that **large systems continually evolve** in a changing environment ⁵³. Two important laws are:
 - *Continuing Change:* Software must continually adapt to new requirements or it becomes obsolete ⁵³. For example, browsers are constantly updated as web standards evolve.
 - *Increasing Complexity:* As software is changed, its internal structure tends to degrade unless effort is made to simplify or refactor ⁵⁴. This is why preventive maintenance is needed. (Other laws note that development effort doesn't necessarily speed up with more resources and that there's a limit to adding new functionality without incurring more faults ⁵⁵.) In exams, Lehman's laws justify why maintenance is a normal, required phase of software life.

- **Legacy Systems:** Many organizations have **legacy systems** – old but critical software built on outdated technology ⁵⁶ . Legacy systems often exhibit: high maintenance cost, complex degraded structure, obsolete platforms or languages, scarce documentation, and lack of original developers ⁵⁷ . For example, a decades-old COBOL-based banking application may work reliably but is expensive to update.
- **Issues:** Legacy systems are “business critical” yet hard to change ⁵⁸ . Poor or missing documentation and outdated practices mean every modification risks new faults ⁵⁹ . Often the “legacy dilemma” arises: the system must keep running, but replacing it entirely is costly and risky ⁶⁰ .
- **Example:** An airline’s reservation system from the 1980s may be in COBOL on mainframes. It still works but new requirements (e.g. web interface) require bridging old and new tech. The maintenance team must deeply understand the legacy code or else the high maintenance cost continues ⁵⁹ .
- **Factors Affecting Maintenance:** Maintenance cost and difficulty are influenced by many factors:
 - **User Requirements:** Frequent or unclear change requests increase effort. Users may lack maintenance context and demand impractical changes ⁶¹ ⁶² .
 - **Organizational Environment:** Changes in business processes or policies require corresponding software updates ⁶³ . Market competition can force rapid changes.
 - **Operational Environment:** Changes in hardware, OS, or third-party software (e.g. new database versions) can force maintenance of the application ⁶⁴ .
 - **Software Product Attributes:** Poor documentation, complex or tangled code structure, and high program complexity increase maintenance effort ⁶⁵ . Conversely, modular, well-documented systems are easier to update.
 - **Maintenance Team:** Turnover, lack of domain expertise or coordination can hinder maintenance. New team members need time to understand the system.
- **Maintenance Process:** Software maintenance follows a cycle of request → analyze → implement → test (often called the maintenance life cycle). Important aspects include: capturing true user requirements (often clarified only in hindsight) and carefully testing fixes (since late-detected bugs are costly to fix) ⁶⁶ . Maintaining quality means balancing urgent fixes with sustainable changes (e.g. not hacking a quick fix that makes future work harder).
- **Deep Takeaways:** Maintenance is an inevitable, ongoing phase. It accounts for the majority of a software’s life-cycle cost. Key exam points include the four maintenance categories (corrective, adaptive, perfective, preventive) and the idea that maintainability should be built in (modularity, good docs). The Lehman laws remind us to plan for change. Finally, recognize that a well-maintained system remains useful and competitive, whereas neglecting maintenance leads to outdated or failed software (just as a car breaks down without service). Regular refactoring, clear documentation, and automated regression testing greatly ease maintenance.

Sources: Key definitions and principles are drawn from the course material in Unit 4 ⁶ ³² ⁴⁷ . Additional specifics on planning, debugging, and maintenance were cited from the provided unit notes ⁷ ⁶⁷ ⁶⁸ .